

Week 9

A – Class Discussion

A – Cloud Computing

1/ What Cloud Computing services does AWS provide?

Amazon Web Services (AWS) offers over 200 services across various categories, allowing businesses and individuals to access computing power, storage, and other IT resources on demand. Key services include:

- **Compute:** Services like **Amazon Elastic Compute Cloud (EC2)** provide virtual servers (instances) that can be resized and configured to run applications. **AWS Lambda** is a **serverless** compute service that allows you to run code without provisioning or managing servers.
- **Storage:** **Amazon Simple Storage Service (S3)** provides scalable object storage for data, backups, and applications.
- **Databases:** **Amazon Relational Database Service (RDS)** simplifies the setup and management of relational databases, while **Amazon DynamoDB** is a fast, flexible NoSQL database service.
- **Networking and Content Delivery:** Services like **Amazon Virtual Private Cloud (VPC)** allow you to create an isolated section of the AWS cloud, and **Amazon CloudFront** is a content delivery network (CDN) that delivers data with low latency.
- **Machine Learning:** **Amazon SageMaker** helps developers build, train, and deploy machine learning models.

2/ How does AWS provide Cloud Computing services?

AWS provides cloud computing services by owning and maintaining a vast global network of secure data centers.

- Instead of buying, owning, and maintaining physical hardware, customers can access these resources on a pay-as-you-go basis over the internet.

- This model allows organizations to acquire IT resources on demand, scale them up or down as needed, and pay only for what they use.
- AWS manages the underlying infrastructure, including hardware, operating systems, and security, freeing customers from these time-consuming and expensive tasks.

3/ What are the risks of using AWS?

While AWS provides a secure and reliable platform, the primary risks often stem from user-side misconfigurations and management errors.

The AWS Shared Responsibility Model states that AWS is responsible for the security of the cloud, while the customer is responsible for security in the cloud. The main risks include:

- **Identity and Access Management (IAM):** Incorrectly configured permissions can grant unauthorized users or services access to sensitive data. Overly broad permissions and not using multi-factor authentication (MFA) are common mistakes.
- **Misconfigured Storage Buckets:** Publicly accessible **Amazon S3** buckets are a leading cause of data breaches. If they are not configured correctly, they can expose sensitive information to the internet.
- **Insufficient Monitoring:** A lack of proper logging and monitoring can lead to security incidents going unnoticed, delaying containment and recovery.
- **Unpatched Vulnerabilities:** Failing to patch applications and operating systems on customer-managed instances (like EC2) can leave them vulnerable to attack.

B – Load Balancing

1/ What facilities does load-balancing give in Cloud Computing?

Load balancing in cloud computing distributes incoming network traffic across multiple servers, preventing any single server from becoming overwhelmed. This provides several key facilities:

- **Increased Availability:** By spreading traffic, a single server failure won't take down the entire application. The load balancer automatically redirects traffic to healthy servers.

- **Improved Performance:** It ensures efficient resource utilization by preventing a single server from being overworked, leading to faster response times and a better user experience.
- **Scalability:** Load balancing works in conjunction with auto-scaling to manage increased traffic by distributing it among newly added servers.
- **Resilience and Disaster Recovery:** It can redirect traffic to servers in different geographical regions in the event of a regional outage or natural disaster, ensuring business continuity.

2/ What is a load-balancing policy?

A load-balancing policy is the specific algorithm or rule a load balancer uses to determine which server receives the next incoming request. Different policies are used for different needs:

- **Round Robin:** This is the simplest policy, distributing requests sequentially to each server in a rotating order.
- **Least Connection:** The load balancer sends the new request to the server with the fewest active connections. This is effective for servers with varying processing times.
- **Least Response Time:** This policy directs traffic to the server that has the lowest average response time and the fewest active connections, optimizing for performance.
- **IP Hash:** The load balancer uses the client's IP address to create a hash, ensuring that the same client is always directed to the same server. This is useful for applications that require "sticky sessions."

C – Auto-Scaling

1/ What facilities does auto-scaling give in Cloud Computing?

Auto-scaling is a cloud computing feature that automatically adjusts the number of computing resources to match the current demand. It provides these facilities:

- **Cost Optimization:** You only pay for the resources you use. During periods of low traffic, auto-scaling reduces the number of running instances, cutting costs.

- **Improved Performance:** It ensures that your application always has enough capacity to handle traffic spikes, maintaining consistent performance and preventing downtime.
- **High Availability and Fault Tolerance:** If an instance fails, auto-scaling can automatically replace it with a new one, ensuring your application remains available.
- **Automation:** Auto-scaling removes the need for manual monitoring and resource provisioning, freeing up IT staff for other tasks.

2/ What is an auto-scaling policy?

An auto-scaling policy is a set of rules and conditions that define when and how an auto-scaling group should scale. These policies dictate when new instances should be added (scale out) or terminated (scale in). Common types of policies include:

- **Target Tracking Scaling:** This is a simple and effective policy where you define a target value for a specific metric (e.g., CPU utilization should stay around 50%). The policy automatically adjusts the number of instances to maintain this target.
- **Simple Scaling:** This policy adds or removes a specific number of instances in response to an alarm. For example, "if average CPU utilization is above 75% for 5 minutes, add 2 instances."
- **Step Scaling:** A more advanced version of simple scaling, it allows for a more nuanced response by adding or removing a different number of instances based on how far the metric exceeds the threshold.
- **Scheduled Scaling:** This policy is used for predictable traffic changes, such as an increase in web traffic every Monday morning. You can set the number of instances to scale at specific times.

B – Group Activity

Tasks

For an IoT health application that has the following parts:

- Client that collects health data
- API that allows data to be uploaded from the client and store in a MongoDB Database.
- A MongoDB Database.
- A microservice which calculates the average heartrate of a user.
- A microservice which calculates the yearly fitness of a user.
- A microservice that can provide the fitness level for all the runners in a marathon.
- A microservice which will detect when individuals are having health issues.

Think about the use of Cloud Computing and answer the following questions.

1. What parts of this application could be replicated on Cloud?

All parts of the IoT health application could be replicated on the cloud, leveraging a model like

Platform as a Service (PaaS) or Infrastructure as a Service (IaaS).

- **API and Microservices:** The API and all four microservices can be deployed as independent services on a cloud platform. Using a

microservices architecture on the cloud allows each service to be developed, deployed, and scaled independently. This is beneficial because the traffic patterns and resource requirements for each microservice are different. For example, the microservice for detecting health issues might have infrequent but critical spikes in demand, while the yearly fitness calculation might be a background task run periodically.

- **MongoDB Database:** Instead of setting up and managing a MongoDB server on a local machine, you could use a managed database service provided by a cloud provider. This is often called

Database as a Service (DBaaS). Cloud providers handle the patching, backups, and scaling of the database, freeing up developers to focus on application logic.

2. What Load-Balancing policy would you use?

For this application, a Least Connection load-balancing policy is the most effective choice.

- The different microservices have varying computational loads.
- For instance, the microservice that calculates the average heart rate might process requests very quickly, while the one that calculates yearly fitness for a user is computationally intensive and takes longer.
- Using a Round Robin policy could send a new, quick request to a server that is still busy with a long-running task, leading to delays.
- The Least Connection policy, however, directs the new request to the server that has the fewest active connections, regardless of what type of task it's handling.
- This ensures a more balanced distribution of the workload and optimizes overall performance by preventing any single instance from becoming a bottleneck.

3. What Auto-Scaling policy would you use?

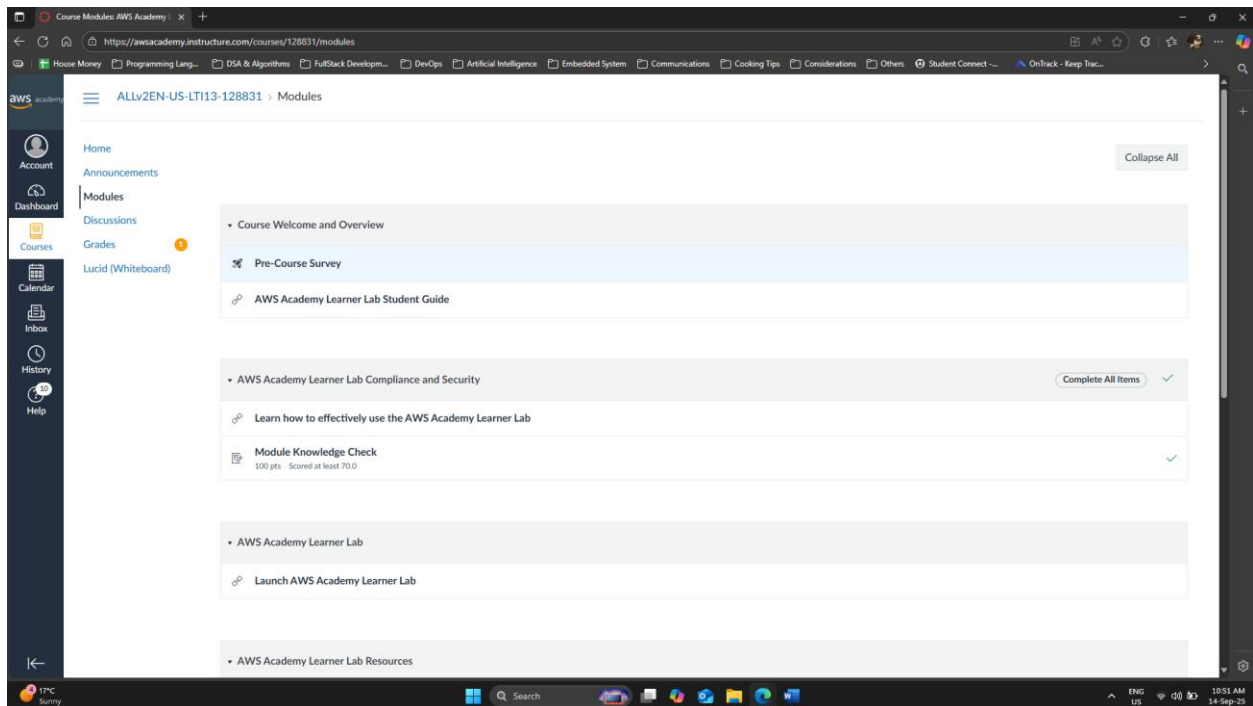
A Target Tracking auto-scaling policy is the most suitable choice for this application.

- This policy is ideal for handling the unpredictable and variable demand of an IoT health application.
 - For example, during a marathon event, the microservice that provides the fitness level for all runners would experience a massive spike in traffic.
 - Rather than manually predicting this surge and scaling up instances, a target tracking policy automatically adjusts the number of instances to keep a key metric, like CPU utilization, at a predefined target level (e.g., 60%).
 - It reacts to changes in real-time.
 - This approach ensures that the application maintains optimal performance during peak loads while also preventing over-provisioning and reducing costs during periods of low activity.

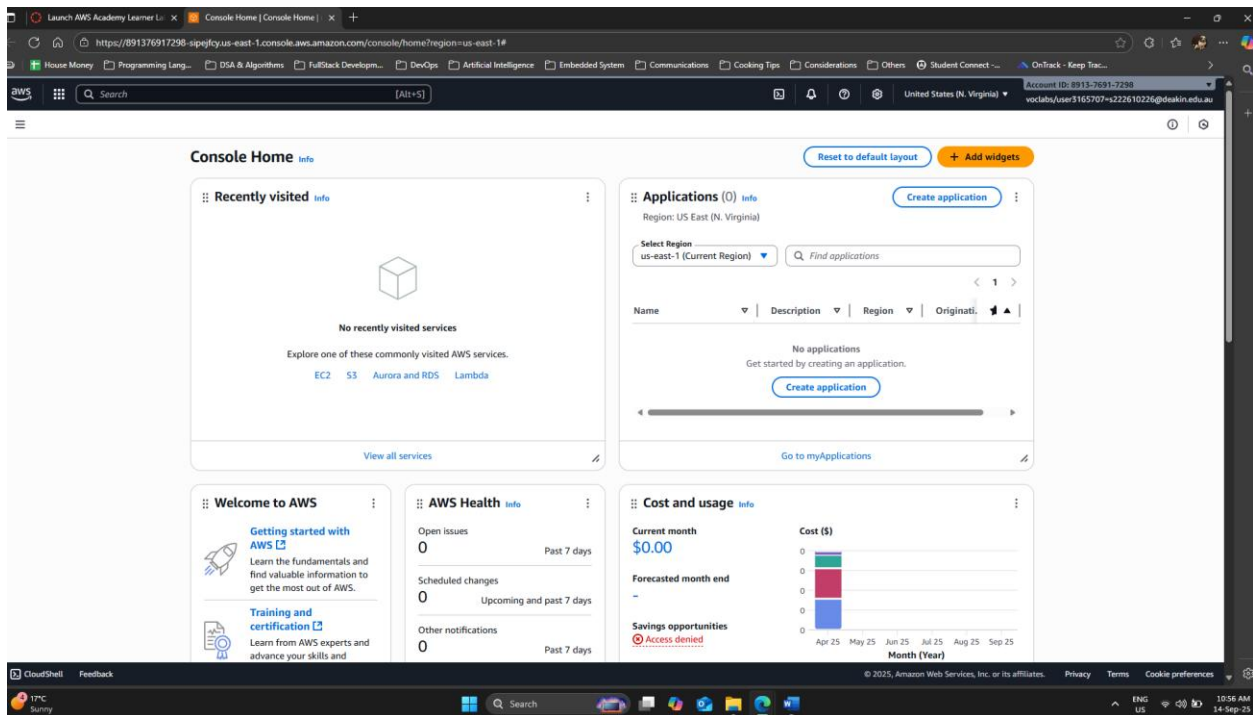
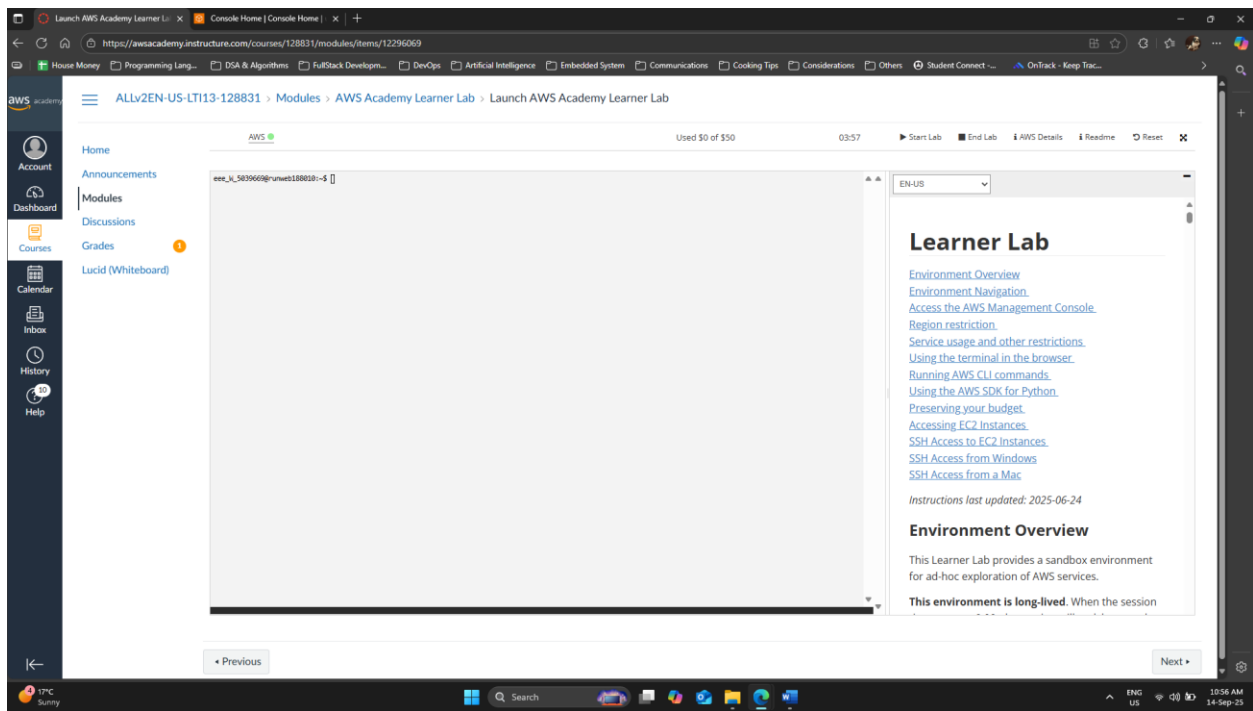
- User will define a metric (e.g., average CPU utilization) and a target value for your microservice auto-scaling group.
 - The auto-scaling policy then monitors this metric. If the average CPU utilization of the instances goes above the target, the policy automatically launches new instances to handle the increased load.
 - Conversely, when the load drops and the metric falls below the target, the policy terminates excess instances to save on costs.
 - This automation is crucial for an application with unpredictable usage patterns.

C – Technical Task

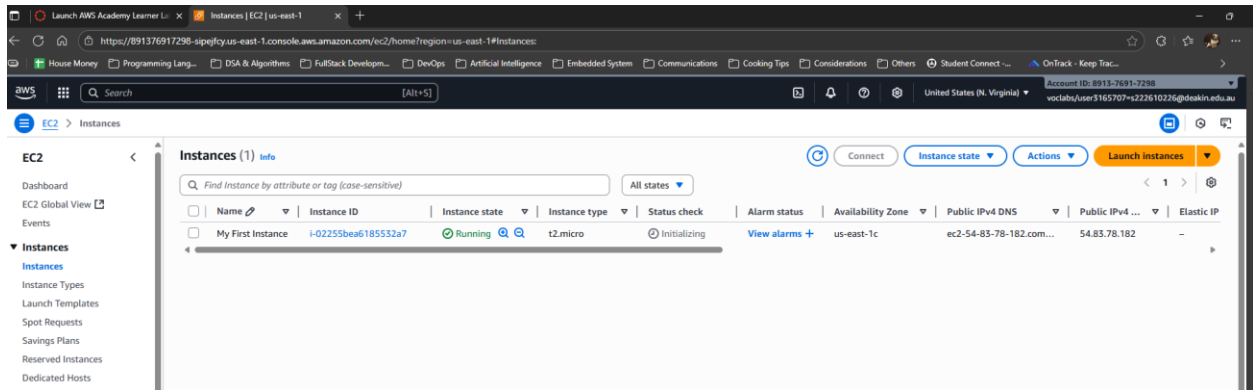
A – Getting an AWS Account & Accessing the AWS Management Console



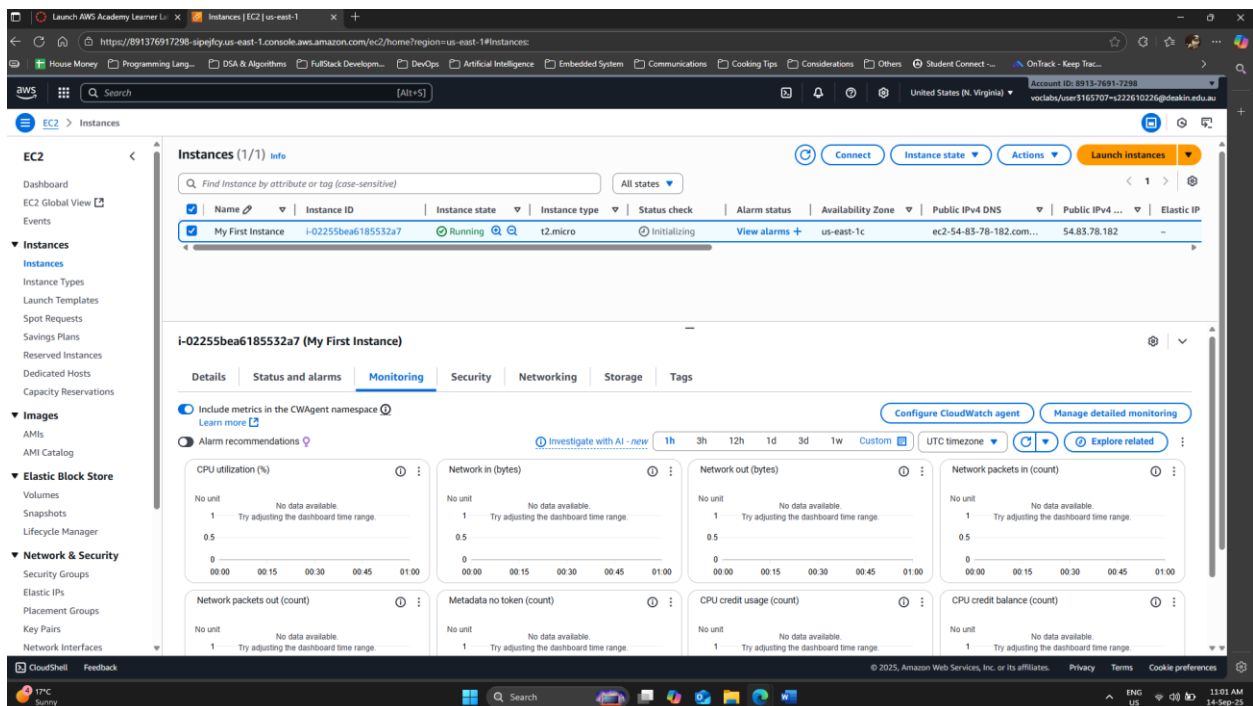
Picture 1 – Completed “Module Knowledge Check”.



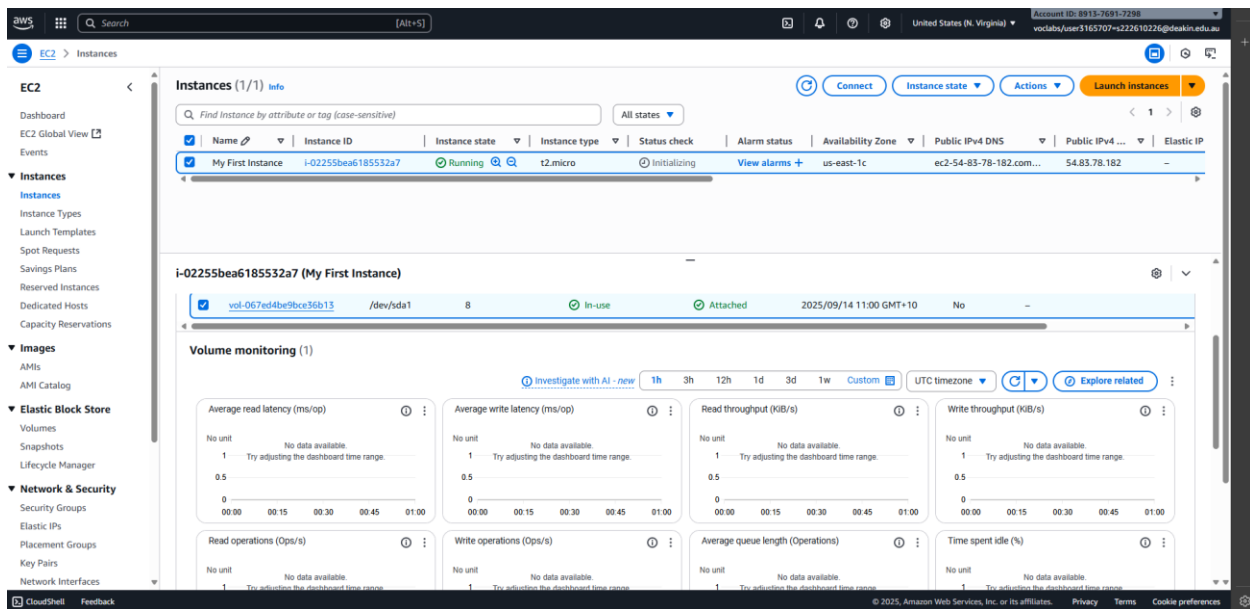
B – Using Ubuntu Linux VM in EC2



Picture 4 – Successfully created an EC2 called “My First Instance”.



Picture 5 – Accessed “Monitoring” tab of the generated EC2 instance.



Picture 6 – Accessed “Volume” tab of the generated EC2 instance.

```
ubuntu@ip-172-31-21-233: ~
user@Legion-7:~$ ssh -i "~/awsLearner_ec2_keyPair.pem" ubuntu@ec2-54-83-78-182.compute-1.amazonaws.com
Load key "/home/user/awsLearner_ec2_keyPair.pem": Permission denied
ubuntu@ec2-54-83-78-182.compute-1.amazonaws.com: Permission denied (publickey).
user@Legion-7:~$ ls -l ~/awsLearner_ec2_keyPair.pem
-r----- 1 root root 1678 Sep 14 10:59 /home/user/awsLearner_ec2_keyPair.pem
user@Legion-7:~$ sudo chown user:user ~/awsLearner_ec2_keyPair.pem
user@Legion-7:~$ chmod 400 ~/awsLearner_ec2_keyPair.pem
user@Legion-7:~$ ls -l ~/awsLearner_ec2_keyPair.pem
-r----- 1 user user 1678 Sep 14 10:59 /home/user/awsLearner_ec2_keyPair.pem
user@Legion-7:~$ ssh -i "~/awsLearner_ec2_keyPair.pem" ubuntu@ec2-54-83-78-182.compute-1.amazonaws.com
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-1011-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

System information as of Sun Sep 14 01:35:43 UTC 2025

System load:  0.08          Processes:      107
Usage of /:   25.6% of 6.71GB Users logged in:  0
Memory usage: 21%          IPv4 address for enx0: 172.31.21.233
Swap usage:   0%

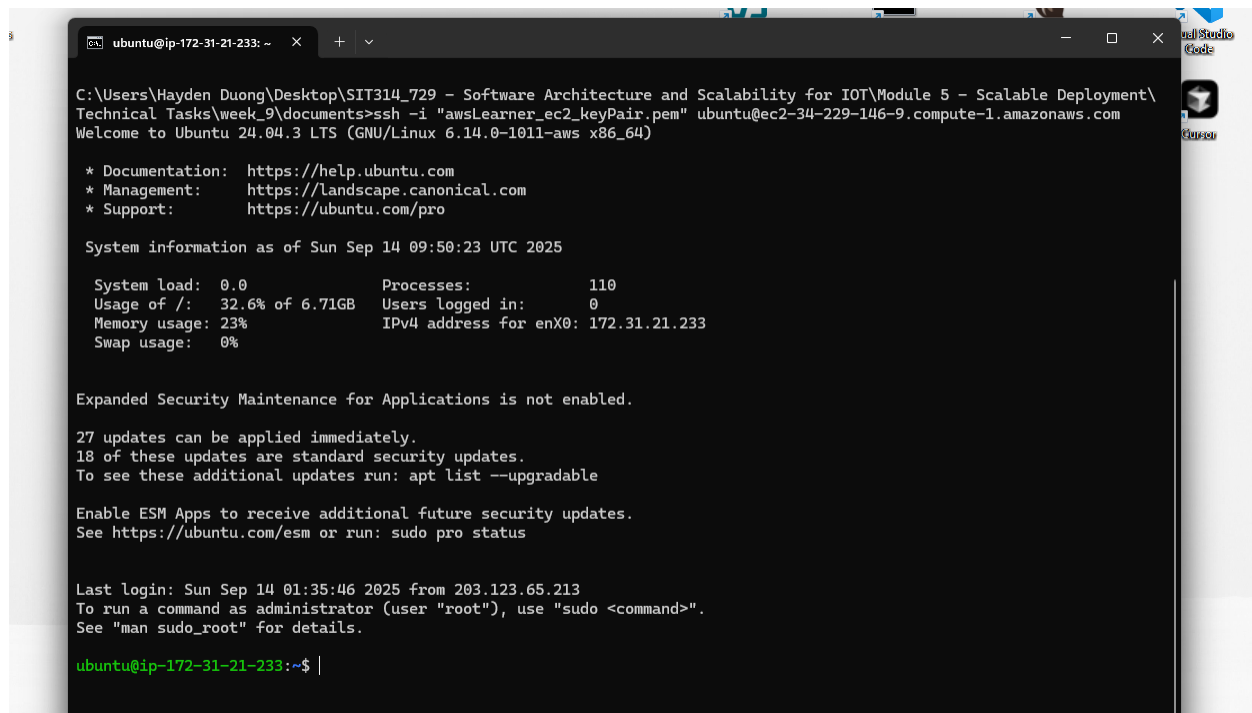
Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status
```

Picture 7 – Edited ownership to the generated keypair & successfully connected to the EC2 instance.

Method 1 - Installed wsl for window, then login to Ubuntu application. Copied the keypair from window – Download folder to `\\wsl$ - \home\user`. Then ran the code above (Picture 7) to configure the ownership to the copied keypair & connected it to EC2 “My First Instance”.



```
ubuntu@ip-172-31-21-233: ~  
C:\Users\Hayden Duong\Desktop\SIT314_729 - Software Architecture and Scalability for IOT\Module 5 - Scalable Deployment\Technical Tasks\week_9\documents>ssh -i "awsLearner_ec2_keyPair.pem" ubuntu@ec2-34-229-146-9.compute-1.amazonaws.com  
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-1011-aws x86_64)  
  
* Documentation:  https://help.ubuntu.com  
* Management:    https://landscape.canonical.com  
* Support:       https://ubuntu.com/pro  
  
System information as of Sun Sep 14 09:50:23 UTC 2025  
  
System load:  0.0      Processes:      110  
Usage of /:   32.6% of 6.71GB   Users logged in:  0  
Memory usage: 23%      IPv4 address for enx0: 172.31.21.233  
Swap usage:   0%  
  
Expanded Security Maintenance for Applications is not enabled.  
  
27 updates can be applied immediately.  
18 of these updates are standard security updates.  
To see these additional updates run: apt list --upgradable  
  
Enable ESM Apps to receive additional future security updates.  
See https://ubuntu.com/esm or run: sudo pro status  
  
Last login: Sun Sep 14 01:35:46 2025 from 203.123.65.213  
To run a command as administrator (user "root"), use "sudo <command>".  
See "man sudo_root" for details.  
  
ubuntu@ip-172-31-21-233:~$
```

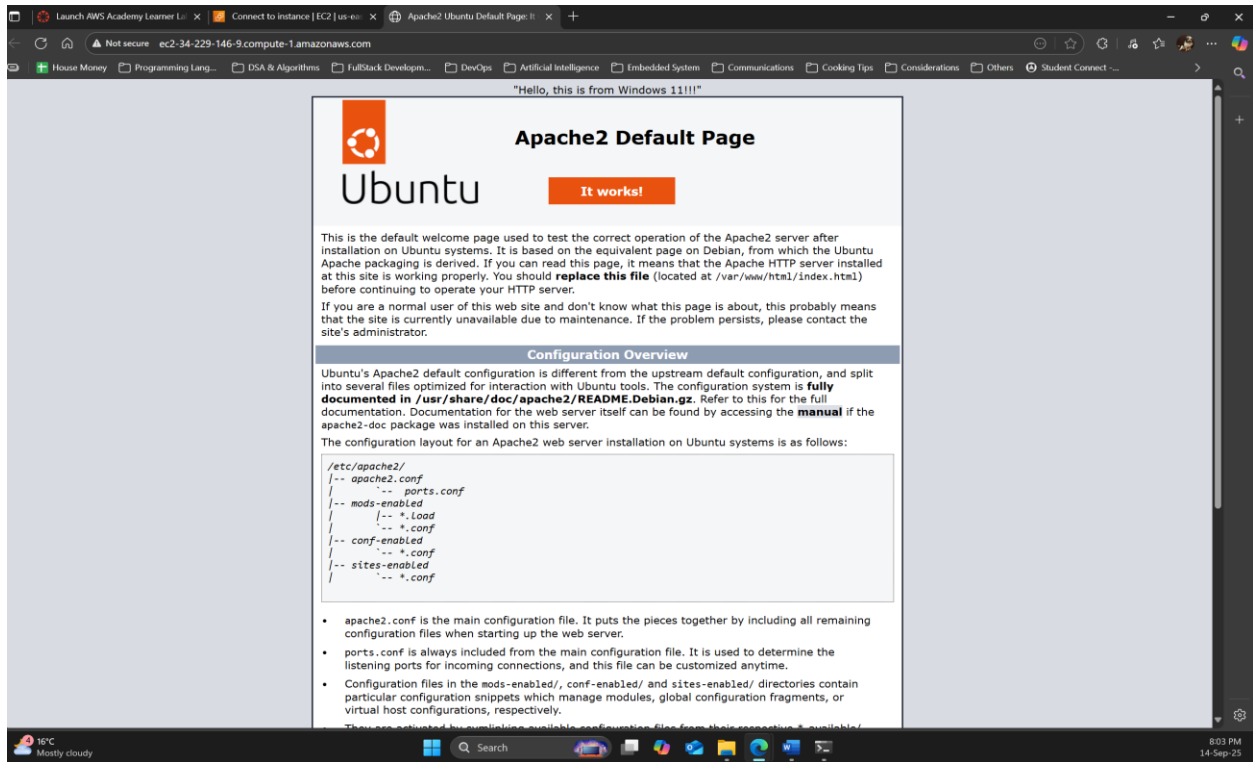
Picture 8 – Connected to EC2 via Windows 11 Command Prompt

Method 2 – Using Window Command Prompt (as OpenSSH is built-in to Windows 11):

Locate the keypair file “.ssh” and changed it properties:

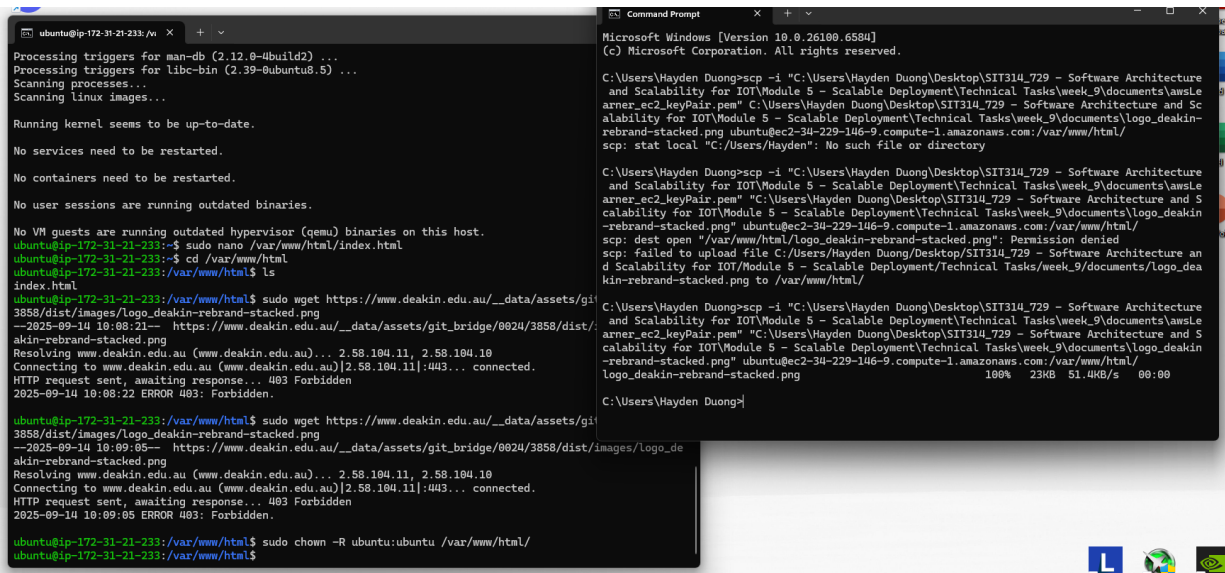
- Properties → Security → Advanced → Clicked “Disable inheritance” → Chose “Remove all inherited permissions from this object”.
- Click “Add” → “Select a principal” → Entered my username on my laptop & Clicked “Check Names” → Clicked “OK”. Then I made sure this user only has “Read” & “Read & execute” and “Apply” + “OK”.

C – Installing and configuring software on Linux

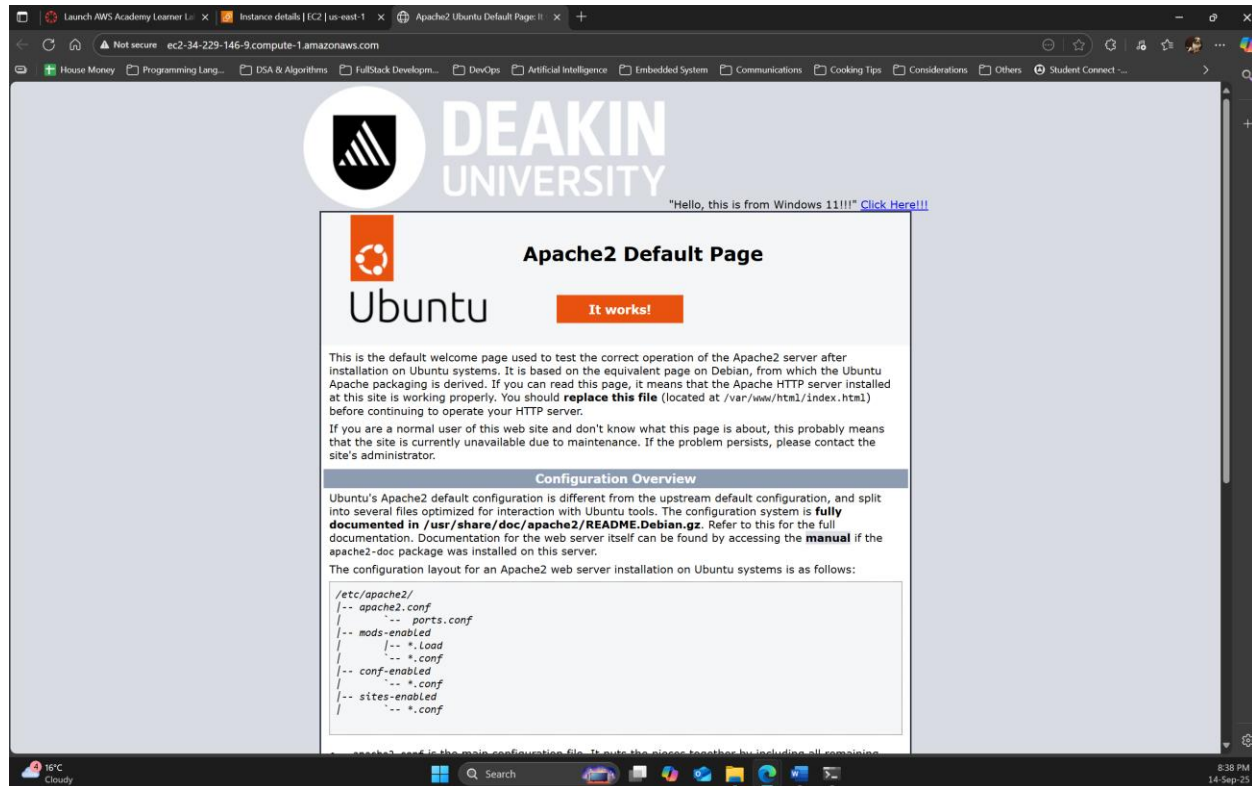


Picture 8 – Successfully displayed the message “Hello, this is from Windows 11!!!” on top on the Apache2 Default Page.

- I followed the lab instructions and modified the “index.html” found inside the Ubuntu OS.
- Accessed the Default Page through <http://ec2-34-229-146-9.compute-1.amazonaws.com/>

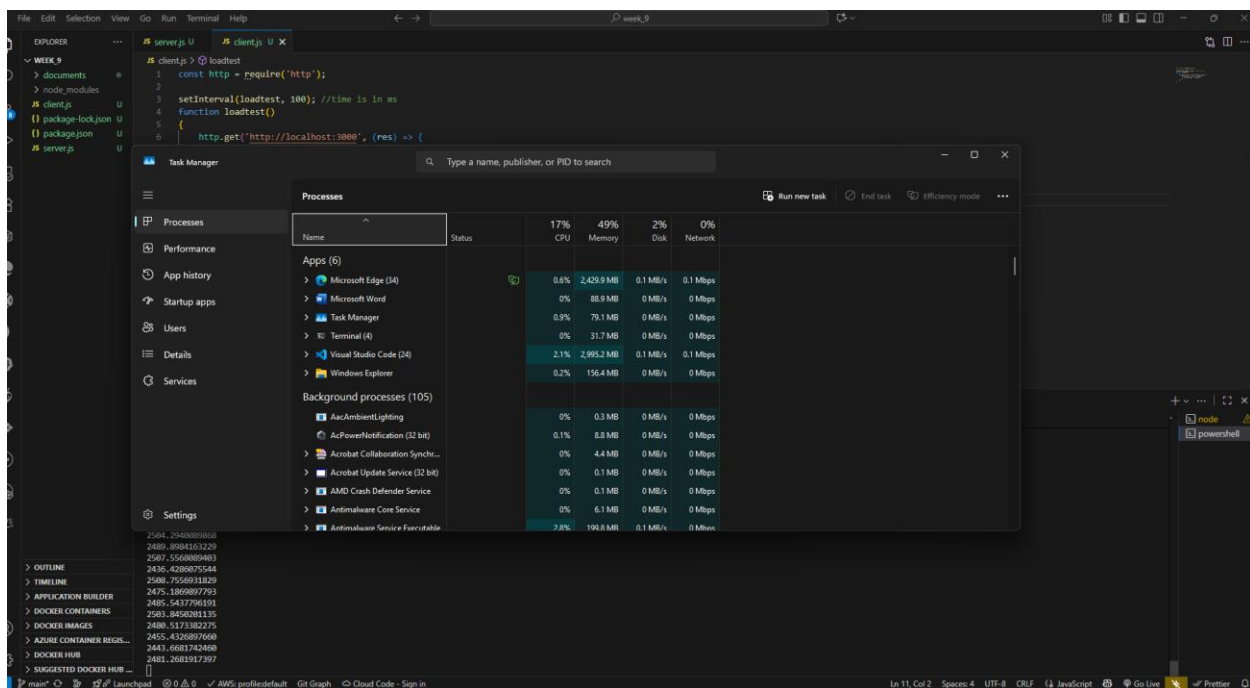


Picture 9 – Successfully “migrate” the manually downloaded .png from local Windows 11 to EC2 instance.

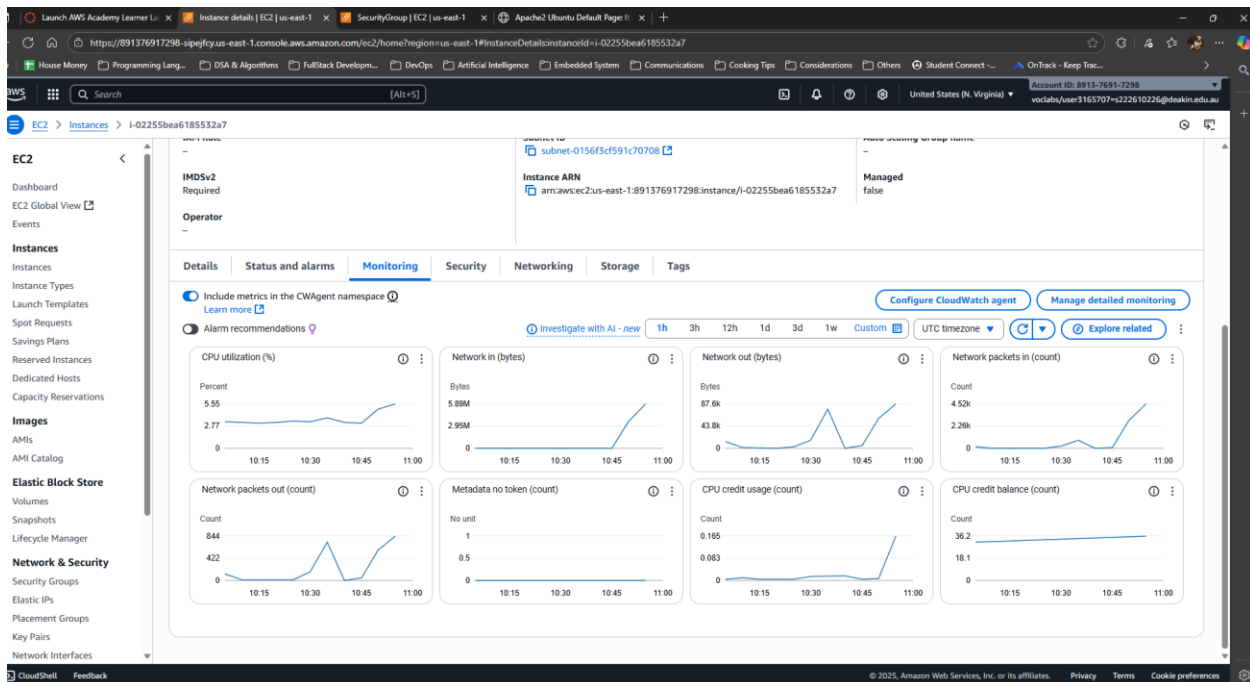


Picture 10 – Successfully displayed Deakin Logo & a link to Deakin University website.

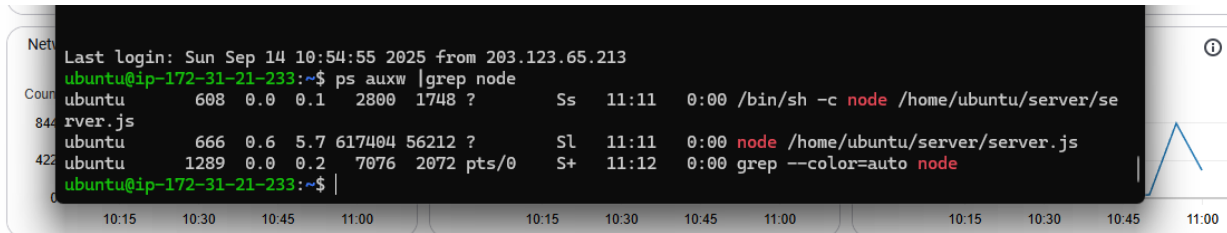
D – Using Load Balancing in EC2



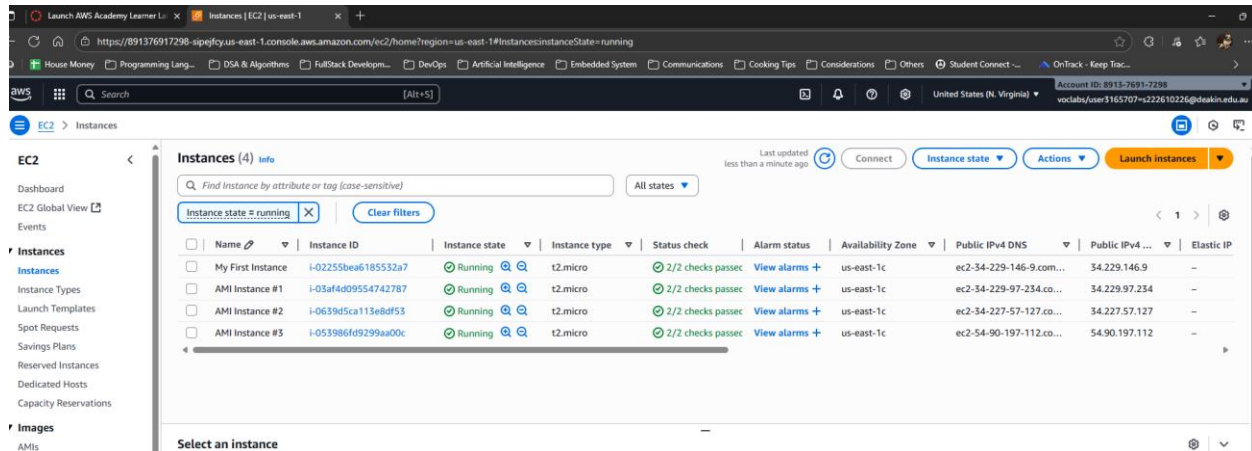
Picture 10 – Windows 11 Task Manager shown high CPU usage of VSCode (from running both node client.js & server.js)



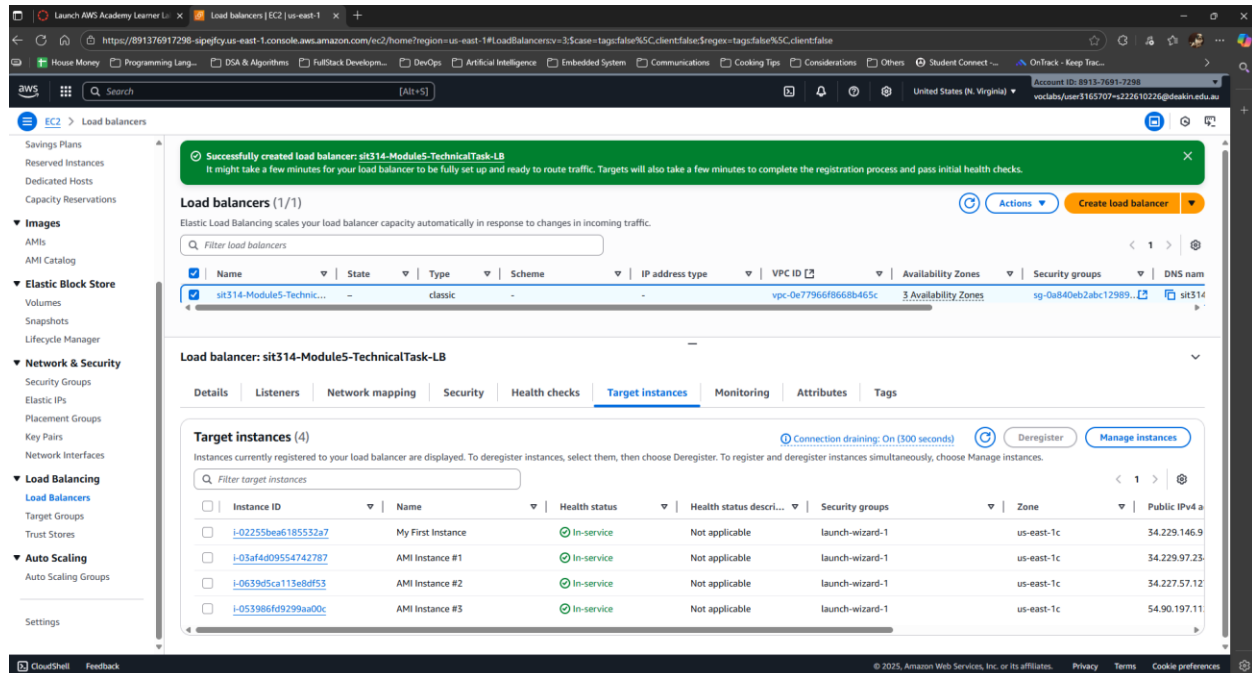
Picture 11 – CPU monitor results from running 3 client.js on local machine.



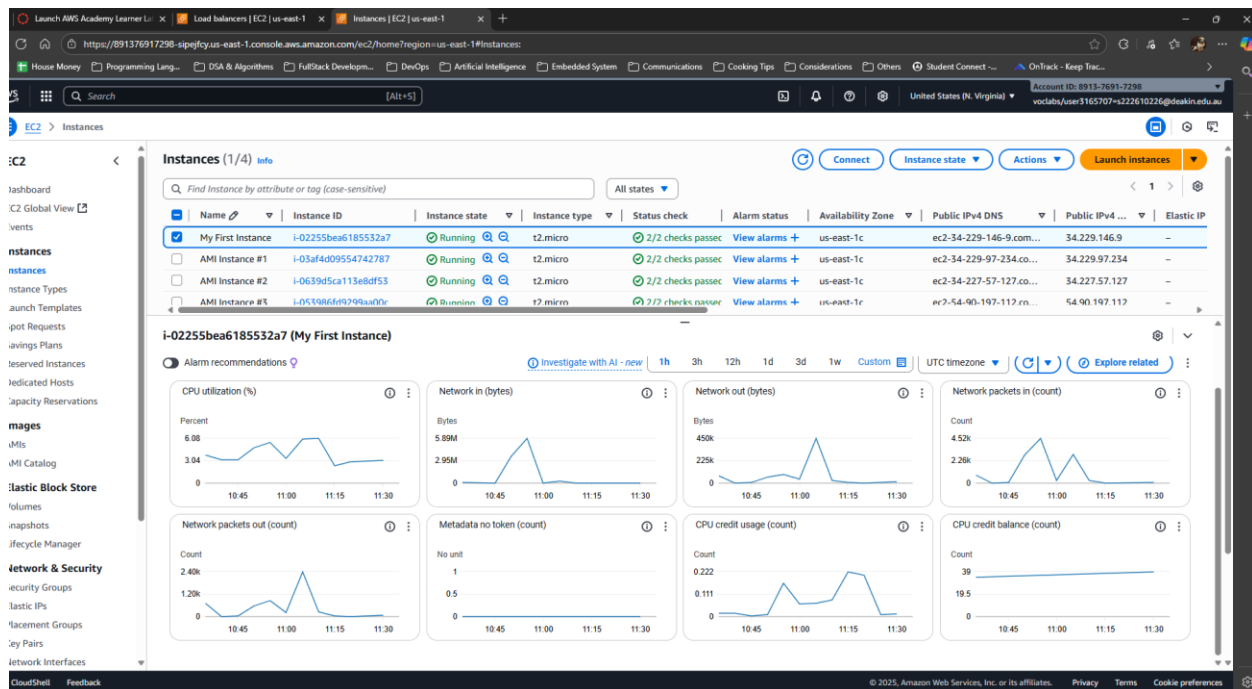
Picture 12 – Node instance is shown running as EC2 is rebooted after edited with crontab -e command.



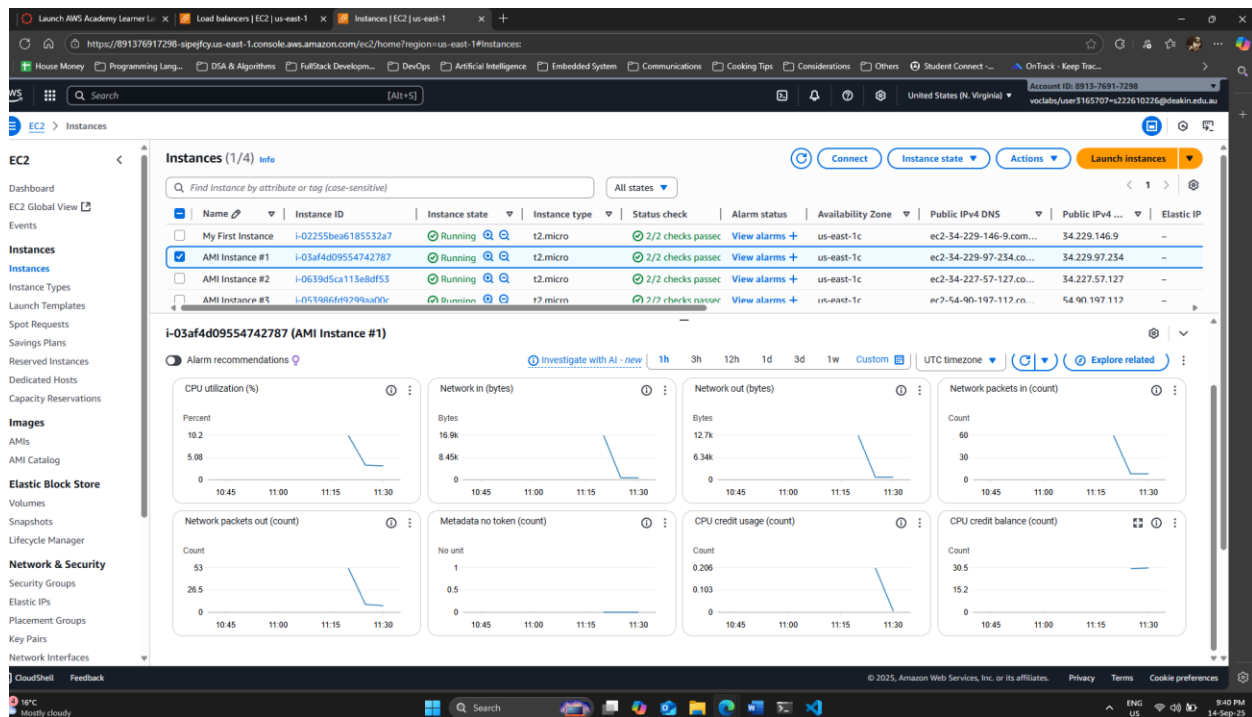
Picture 13 – Total 4 instances (1 original & 3 AMI-generated) are running.



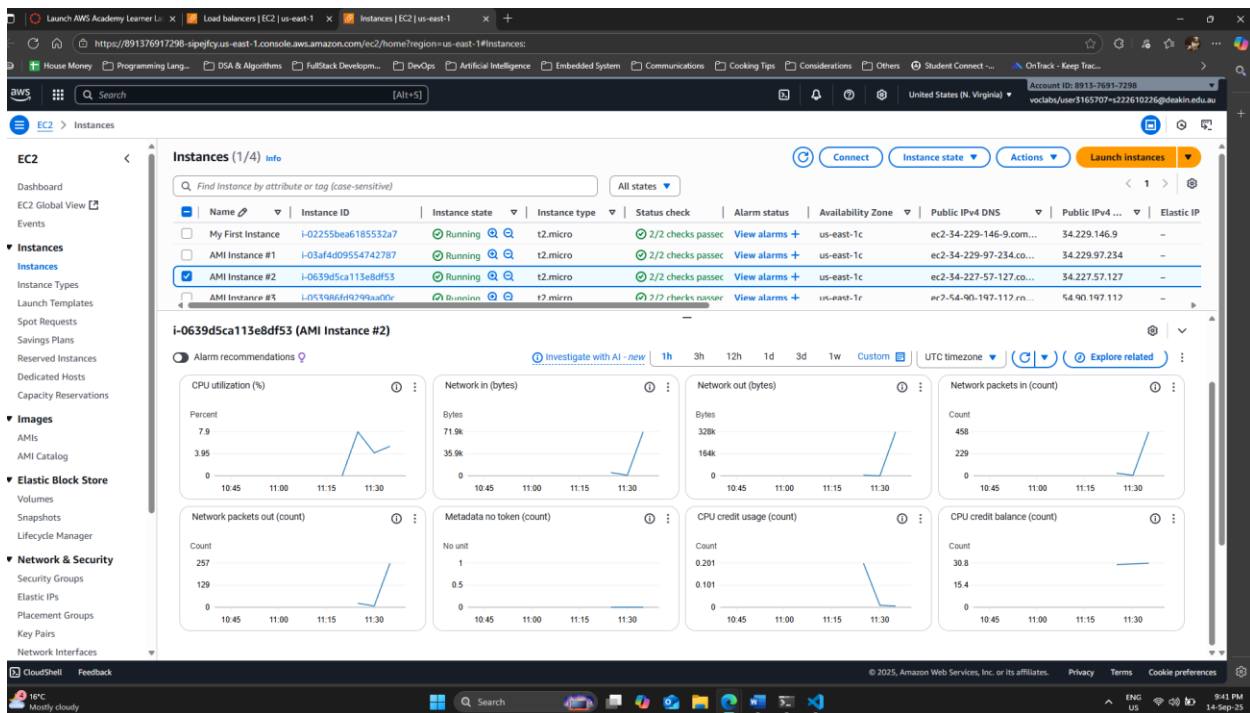
Picture 14 – Load Balancer - 4 target instances are running.



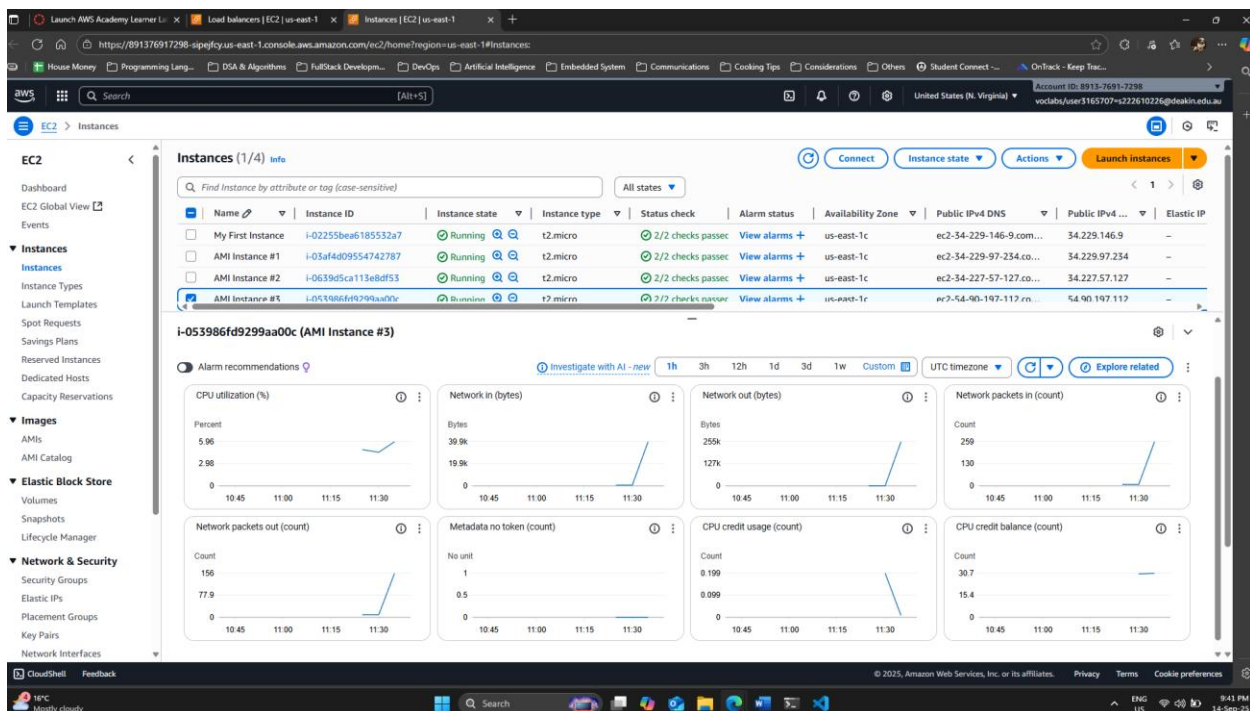
Picture 15 – Monitoring Result from “My First Instance”



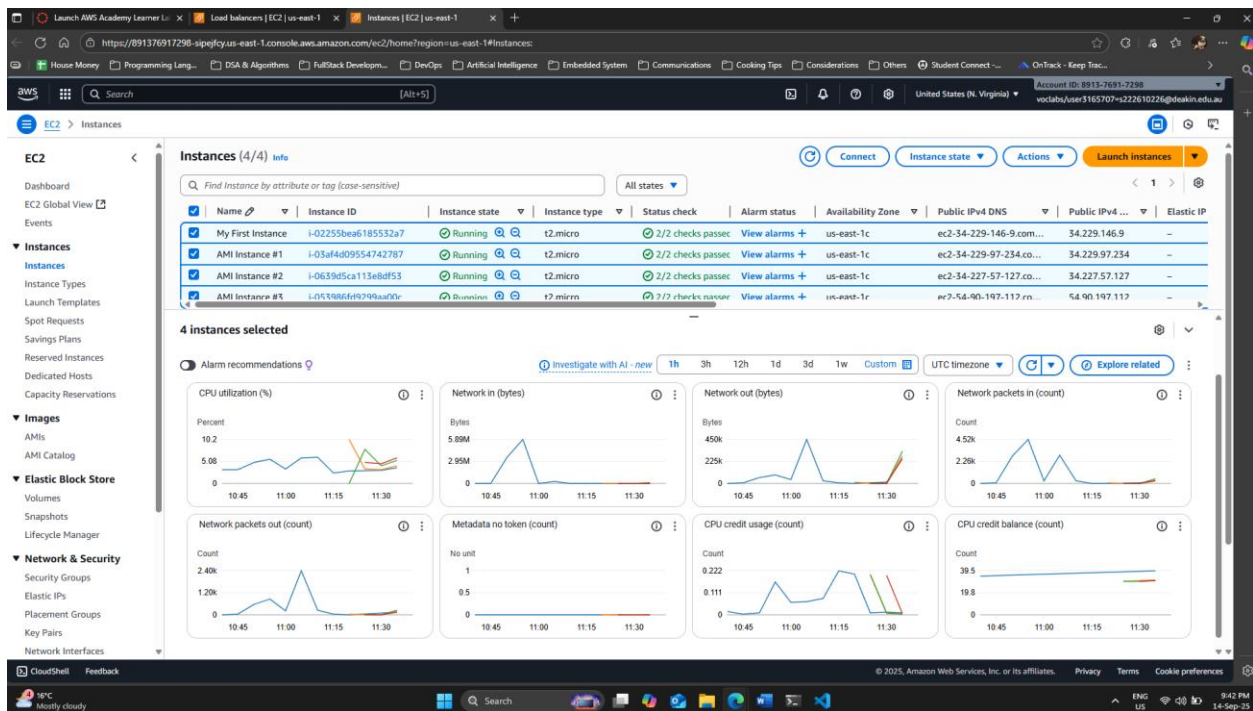
Picture 16 – Monitoring Result from "AMI Instance #1”



Picture 17 - Monitoring Result from "AMI Instance #2"

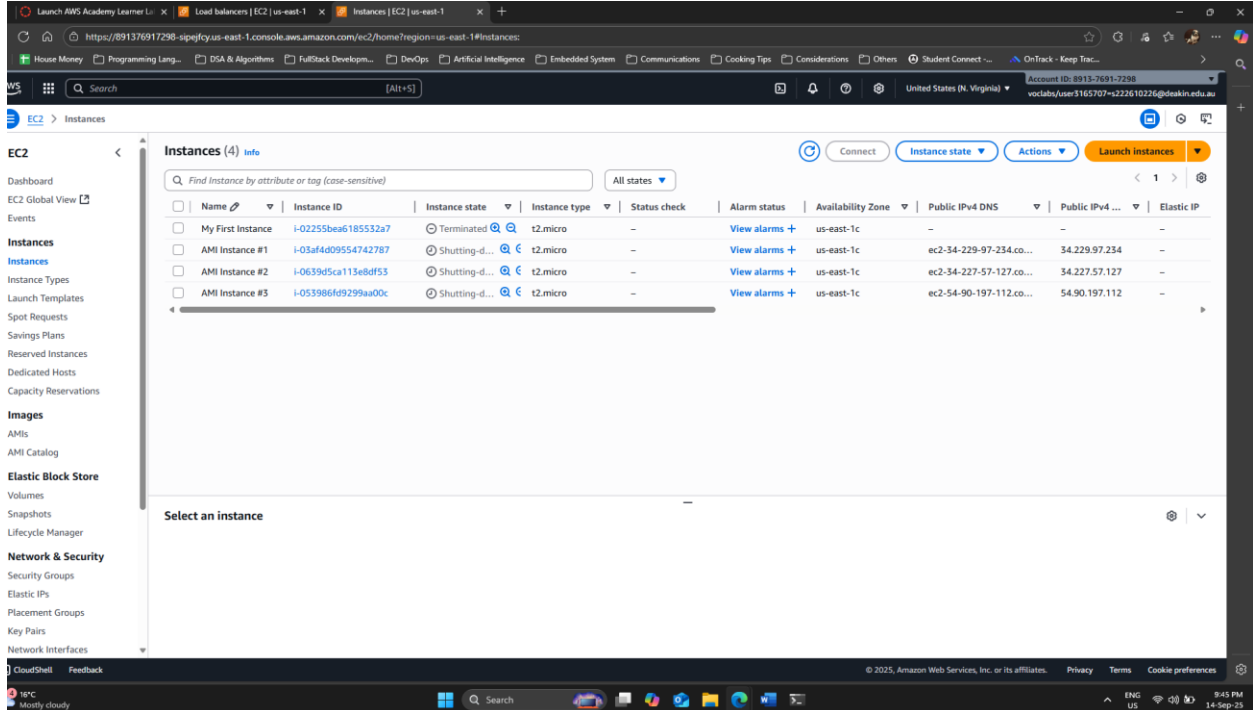


Picture 18 – Monitoring Result from “AMI Instance #3”



Picture 19 – Monitoring Result from all 4 running instances.

E – Clean Up



Picture 20 – Shutting Down and Terminating all 4 running instances.

Week 10

A – Class Discussion

Docker

1 – What is Docker and why would we want to use Docker?

- Docker is a platform for developing, shipping, and running applications in isolated environments called containers.
- It packages an application and all its dependencies—libraries, system tools, code, and runtime—into a container.
- This ensures the application works identically regardless of its environment.
- Thus, the reasons for using Docker are as follows:
 - **Consistency:**
 - Containers guarantee that your application will run the same way in development, testing, and production environments, eliminating the "it works on my machine" problem.
 - **Isolation:**
 - Each container runs as an isolated process, so different applications can coexist on the same host without interfering with each other.
 - This is great for security and stability.
 - **Portability:**
 - A single container image can be deployed on any system that has the Docker engine, from a developer's laptop to a cloud server, making it easy to move applications.
 - **Efficiency:**
 - Containers are lightweight and start up quickly, using fewer resources than traditional virtual machines.
 - This means you can run more applications on a single server.

2 – What is a Docker image, container, engine, registry service?

- A Docker image is a read-only template used to create containers.

- It's a blueprint that contains the application code, a runtime, libraries, environment variables, and configuration files.
 - Can be considered as a class in object-oriented programming.
- A Docker container is a runnable instance of a Docker image.
 - It's the actual application running in its isolated environment.
 - Users can start, stop, move, or delete a container.
- The Docker engine is the core software that runs and manages Docker containers on a host machine.
 - It's a client-server application with a daemon (server), a REST API, and a command-line interface (CLI).
- A Docker registry service, like Docker Hub, is a centralized repository for storing and sharing Docker images.
 - Developers can pull images from the registry to create containers or push their own custom-built images to it.

3 – What is a typical workflow deployment using Docker?

1. **Develop the Application:**

- A developer writes the application code.

2. **Create a Dockerfile:**

- A Dockerfile is a text file with instructions for building a Docker image.
- It specifies the base operating system, adds the application code, installs dependencies, and defines how to run the application.

3. **Build the Image:**

- The developer runs the docker build command, which uses the Dockerfile to create a Docker image.

4. **Push to a Registry:**

- The newly built image is tagged and pushed to a Docker registry (e.g., Docker Hub or a private registry) so it can be accessed by others.

5. **Deploy the Container:**

- In a production environment, the deployment server or orchestration tool pulls the image from the registry.

6. **Run the Container:**

- The docker run command is used to start a new container instance from the image.
- This container then hosts the running application, ready to serve requests.

Docker Compose, Docker Swarm, and Kubernetes

1 – What are the main functions of Docker Compose, Docker Swarm, Kubernetes?

- Docker Compose is a tool for defining and running multi-container Docker applications.
 - It uses a single YAML file to configure all the application's services (e.g., a web server, a database, and a cache) so users can start them all with a single command.
 - It's mainly used for development and testing on a single host.
- Docker Swarm is Docker's native container orchestration tool.
 - It allows users to create and manage a cluster of Docker hosts (a "swarm") as a single virtual host.
 - It automates the deployment, scaling, and management of containers across multiple machines.
- Kubernetes is an open-source container orchestration system for automating the deployment, scaling, and management of containerized applications.
 - It's a more comprehensive and powerful system than Docker Swarm, designed to handle large, complex production workloads.

2 – What are the pros and cons of Docker Swarm & Kubernetes?

Feature	Docker Swarm	Kubernetes
Pros	Simplicity:	Scalability and Extensibility:

Feature	Docker Swarm	Kubernetes
	• It's integrated into the Docker ecosystem, making it easy to set up and use. • It has a lower learning curve than Kubernetes. Performance: • Swarm's lightweight design and minimal overhead can be faster for certain tasks.	• It is a more robust, feature-rich, and scalable platform, capable of managing thousands of nodes. Community and Ecosystem: • It has a massive community and a vast ecosystem of tools and services. Advanced Features: • Offers advanced features like auto-scaling, self-healing, and a rich set of deployment strategies.
Cons	Fewer Features: • Lacks the advanced features and comprehensive management capabilities of Kubernetes. Community: • The community and tooling ecosystem are smaller compared to Kubernetes. Less Robust: • It is generally considered less resilient and production-ready for highly complex, large-scale deployments.	Complexity: • It has a very steep learning curve. • Configuring and managing a Kubernetes cluster is complex and requires specialized knowledge. Resource Intensive: • A Kubernetes cluster can be resource-intensive, requiring more computing resources and overhead to run. Setup: • The initial setup is more involved and complex than setting up a Docker Swarm.

B – Group Activity

Tasks

For the same IoT health application that we defined last week, which contains:

- Client that collects health data
- API that allows data to be uploaded from the client and store in a MongoDB Database.
- A MongoDB Database.
- A microservice which calculates the average heartrate of a user.
- A microservice which calculates the yearly fitness of a user.
- A microservice that can provide the fitness level for all the runners in a marathon.
- A microservice which will detect when individuals are having health issues.

Think about the use of containers to structure and implement the deployment of IoT applications.

1 - What parts would make most sense to be containerised?

Each of the following should be its own container:

- **API:**
 - This service handles data uploads from the client and stores them in the database.
 - Containerizing it ensures that the API, along with its specific dependencies and runtime environment, is isolated from other services.
- **MongoDB Database:**
 - Although databases can be run directly on a host, containerizing MongoDB is a common practice for development and production.
 - It simplifies deployment, versioning, and management.
- **Microservices:**
 - Each of the four microservices should be its own container.
 - This is the very essence of microservice architecture: each service is a self-contained unit that can be developed, deployed, and scaled independently.

- This includes the microservice for calculating average heart rate, the one for calculating yearly fitness, the one for providing fitness levels for marathon runners, and the one for detecting health issues.
- **Client:**
 - The client that collects health data is another good candidate for containerization, especially if it's a backend service or a part of a distributed system.
 - This ensures consistency in its runtime environment.

2 - What base image would you base each container on?

- **API and Microservices:**
 - For services written in common languages like Python, Node.js, Java, or Go, user would choose to use an official base image for that language.
 - For instance, a Python application would use python:3.9-slim, while a Node.js application would use node:16-alpine.
 - Using slim or alpine versions is a best practice to keep image sizes small.
- **MongoDB Database:**
 - The official mongo base image from Docker Hub is the standard choice.
 - This image is maintained by the MongoDB team and is optimized for containerized environments.

3 - Which containers would you autoscale?

Autoscaling is essential for services that experience fluctuating workloads, ensuring this application can handle peak demand without wasting resources during off-peak times.

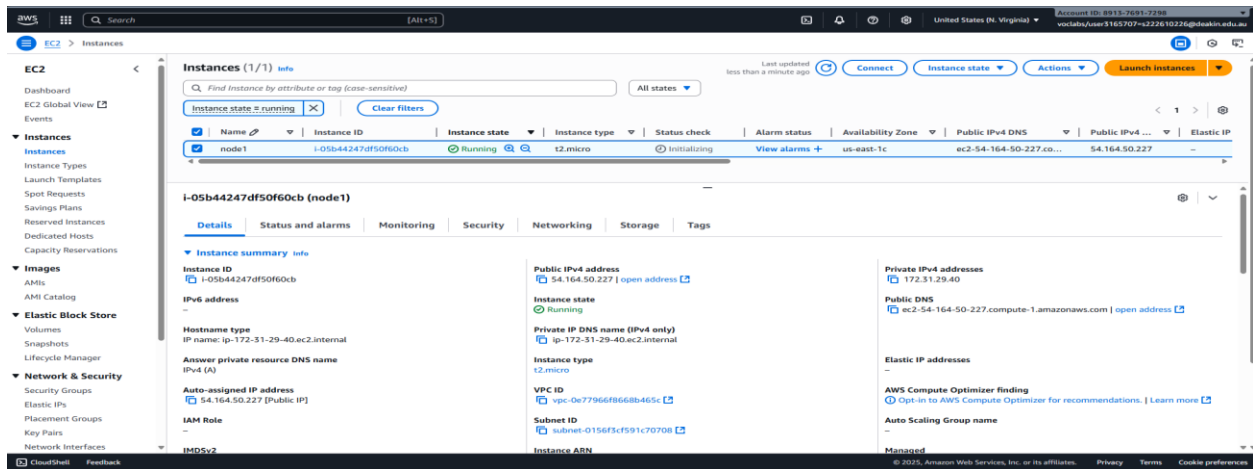
- **API:**
 - This service would be a prime candidate for autoscaling.
 - As more clients upload health data, the API will experience an increase in traffic.
 - Autoscaling allows developers to add more API container instances to handle the load and then scale them down when traffic decreases.
- **Microservice for Detecting Health Issues:**
 - This microservice is likely a critical, real-time service.
 - During events like a marathon or a health emergency, the number of health data points to process would surge.
 - Autoscaling this container ensures that it can keep up with the real-time analysis required to detect issues promptly.
- **Microservice for Marathon Runners:**

- This service would also be a good candidate for autoscaling.
- During a marathon, the number of users and data points would spike, requiring more processing power.
- Autoscaling would ensure the service can handle this temporary high load efficiently.

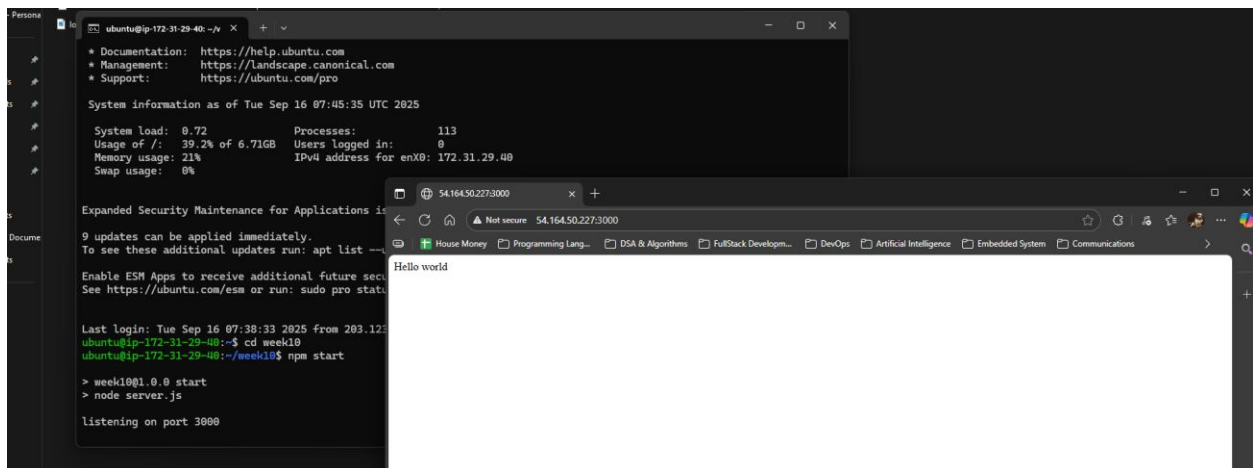
The other microservices (for average heart rate and yearly fitness) might not need to be autoscaled as aggressively, as they could be less time-critical or might run on a schedule, but they could still be considered for autoscaling if they are part of a user-facing, high-traffic feature.

C – Technical Task

Screenshots



Picture 1 – Launched an instance, “node1”, from previous week AMI.



Picture 2 – Accessed to the webpage through “node1” ip4_address:3000.

```
Successfully tagged sit314:latest
ubuntu@ip-172-31-29-40:~/week10$ sudo docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
sit314        latest   8de88f8b4666   12 seconds ago 1.1GB
<none>        <none>    bbfd52b8164   19 minutes ago 923MB
node         20       d1dc1d708aab   12 days ago    1.1GB
node         12       6c8de432fc7f   3 years ago    918MB
```

Picture 3 – Created a new Docker Image (node 20) instead of node-12 version.

```
ubuntu@ip-172-31-29-40:~$ sudo docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
sit314        latest   8de88f8b4666   4 minutes ago 1.1GB
node         20       d1dc1d708aab   12 days ago    1.1GB
node         12       6c8de432fc7f   3 years ago    918MB

ubuntu@ip-172-31-29-40:~$ sudo docker run -p 3000:3000 sit314
> week10@1.0.0 start
> node server.js

Listening on port 3000
npm error path /usr/src/app
npm error command failed
npm error signal SIGTERM
npm error command sh -c node server.js
npm error A complete log of this run can be found in: /root/.npm/_logs/2025-09-16T08_31_04_5752-debug-0.log

ubuntu@ip-172-31-29-40:~$ sudo docker container ls
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                    NAMES
ef01f67afa68   sit314    "docker-entrypoint.s..." 2 minutes ago Up 2 minutes   0.0.0.0:3000->3000/tcp   confident_liskov

ubuntu@ip-172-31-29-40:~$ sudo docker stop ef01f67afa68
ef01f67afa68
```

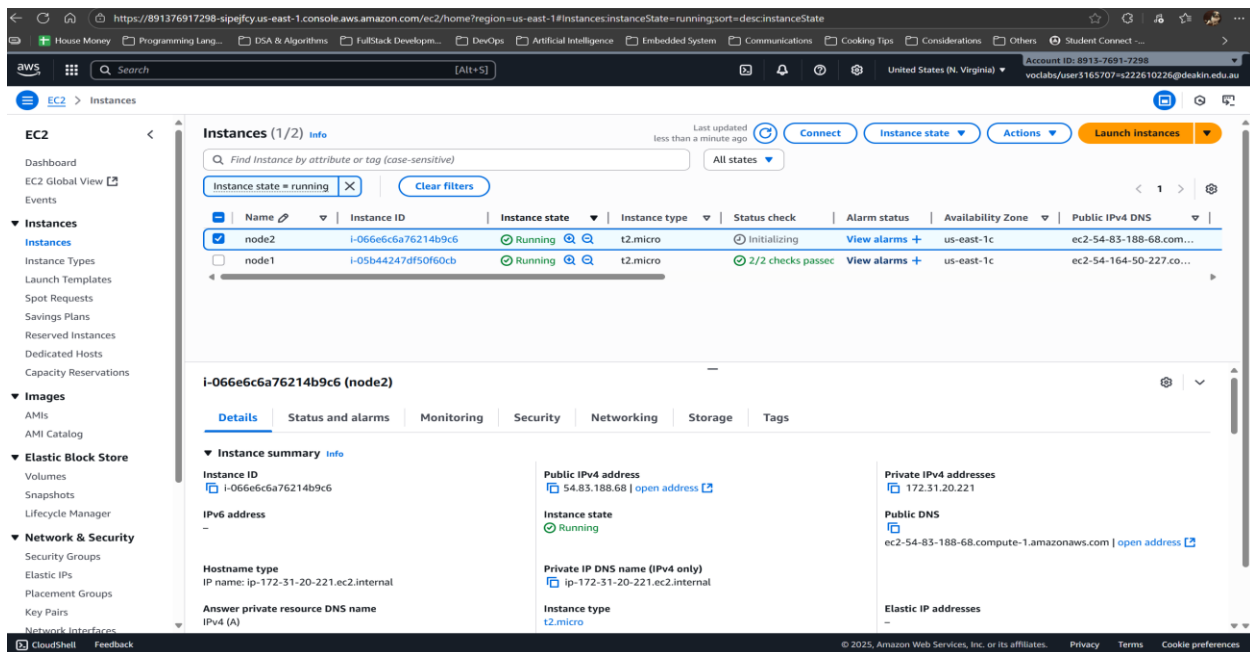
Picture 4 – Stop the running docker in a new terminal (connected to the same EC2).

```
ubuntu@ip-172-31-29-40:~$ sudo docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
sit314        latest   8de88f8b4666   22 minutes ago 1.1GB
node         20       d1dc1d708aab   12 days ago    1.1GB

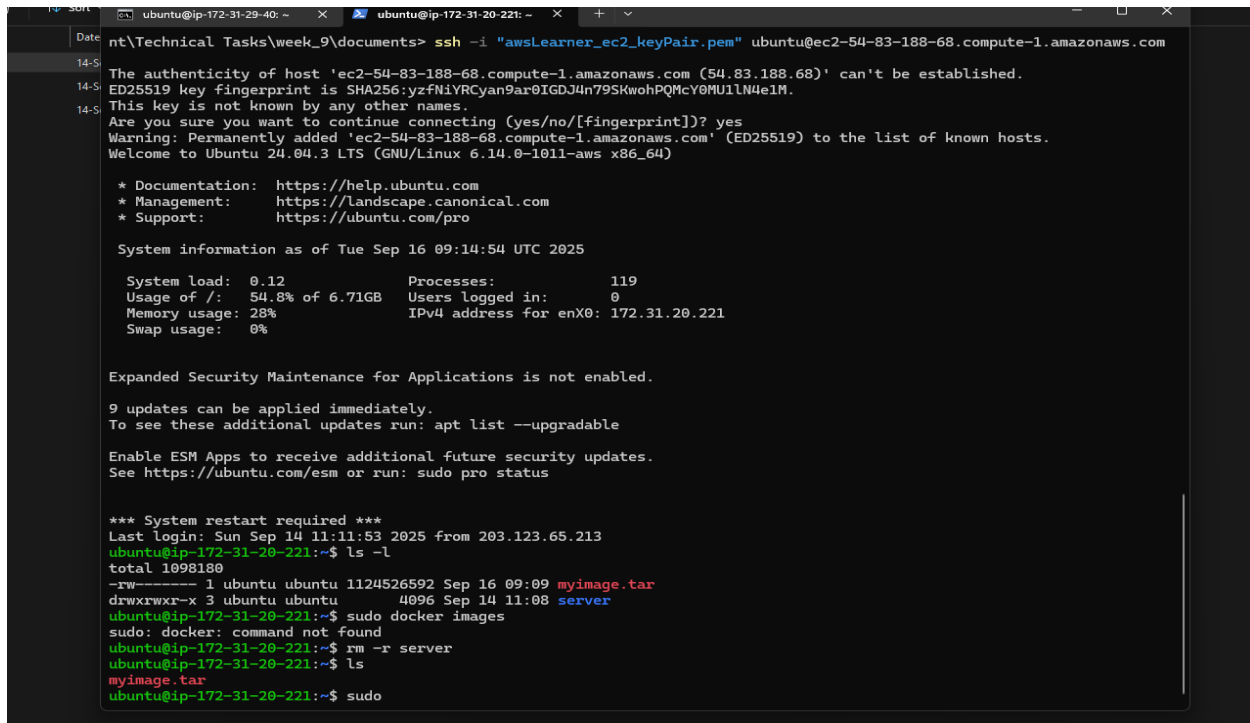
ubuntu@ip-172-31-29-40:~$ sudo docker save -o myimage.tar sit314
ubuntu@ip-172-31-29-40:~$ ls -l
total 1098180
-rw----- 1 root root 1124526592 Sep 16 08:53 myimage.tar
drwxrwxr-x 3 ubuntu ubuntu 4096 Sep 16 08:27 week10

ubuntu@ip-172-31-29-40:~$
```

Picture 6 – Made a copy from docker image sit314.



Picture 7 – Generated a second instance, “node2”, from the same AMI as “node1”.



Picture 8 – Successfully transferred docker image from EC2 “node1” to “node2”.

```
14-S PS C:\Users\Hayden Duong\Desktop\SIT314_729 - Software Architecture and Scalability for IOT\Module 5 - Scalable Deployment\Technical Tasks
\week_9\documents> ssh -i "awsLearner_ec2_keyPair.pem" ubuntu@ec2-54-83-188-68.compute-1.amazonaws.com

Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-1011-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

System information as of Tue Sep 16 09:14:54 UTC 2025

System load:  0.12          Processes:      119
Usage of /:   54.8% of 6.71GB Users logged in:  0
Memory usage: 28%          IPv4 address for enx0: 172.31.20.221
Swap usage:   0%

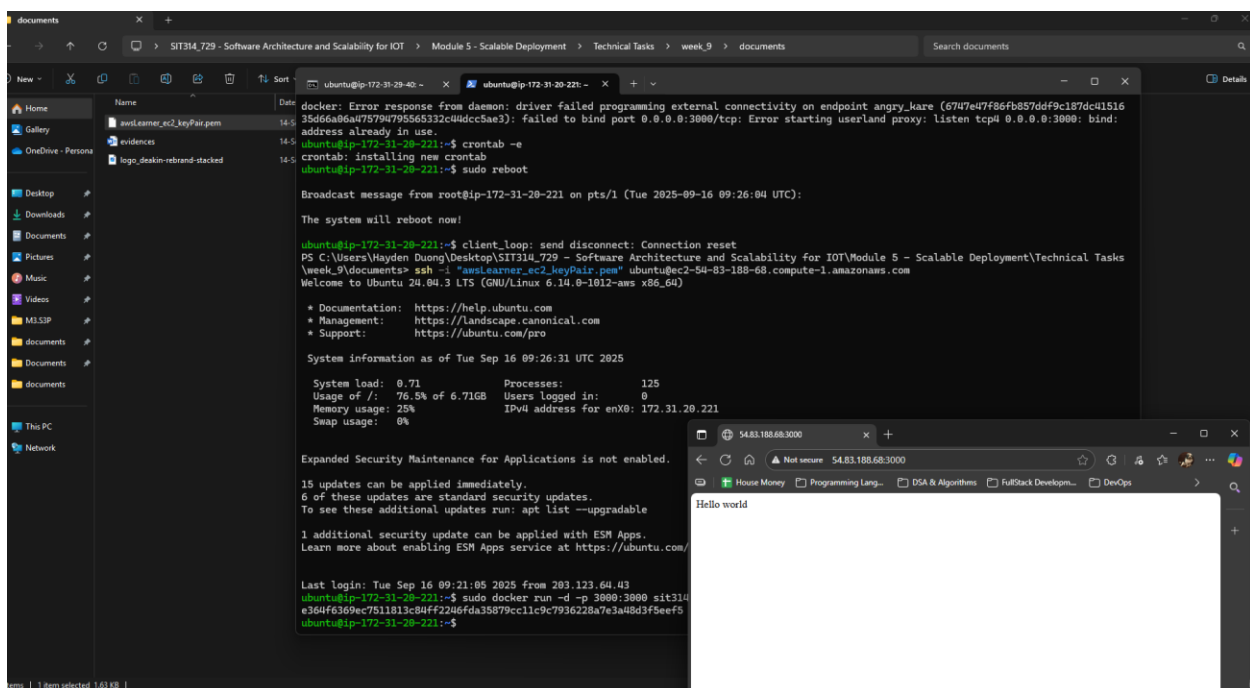
Expanded Security Maintenance for Applications is not enabled.

15 updates can be applied immediately.
6 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

1 additional security update can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

*** System restart required ***
Last login: Tue Sep 16 09:14:55 2025 from 203.123.65.213
ubuntu@ip-172-31-20-221:~$ sudo docker image ls
REPOSITORY TAG IMAGE ID CREATED SIZE
sit314 latest 8de88f8b4666 51 minutes ago 1.1GB
ubuntu@ip-172-31-20-221:~$ sudo docker load -i myimage.tar
Loaded image: sit314:latest
ubuntu@ip-172-31-20-221:~$ sudo docker image ls
REPOSITORY TAG IMAGE ID CREATED SIZE
sit314 latest 8de88f8b4666 53 minutes ago 1.1GB
ubuntu@ip-172-31-20-221:~$ |
```

Picture 9 – The transferred image is loaded onto “node2”.



Picture 10 – Successfully accessed to the “node2” ip_adress:3000.

Module Summary

This module, "Scalable Deployment," covers essential AWS services and Docker for building resilient, high-performance applications. Key takeaways include:

- **AWS EC2, AMI, ELB, and Auto Scaling:** Understanding how virtual servers, standardized images, load balancing, and automatic scaling combine to ensure applications handle varying loads, maintain high availability, and optimize costs.
- **Docker Fundamentals:** Learning to use Dockerfiles to build consistent, portable application images, and how to manage and transfer these containers for streamlined deployments.
- **Linux CLI & Security:** Practical skills in automating application startup (crontab), secure file transfers (scp) with correct permissions (chmod, chown), and troubleshooting common deployment errors.

For a **Software Engineer/Full-stack Dev** (and future **Tech Lead**), this module is crucial. It provides the foundation for building robust cloud-native applications, ensuring consistent environments, optimizing performance, and managing operational complexities efficiently – all vital for successful product delivery and team leadership.